

Terraform diagnostic and repair plan: Harbor Goods

Contents

Summary	1
Scope and method	2
Findings	2
F1. Uploads bucket is public (Critical)	2
F2. Application role can do anything (Critical)	3
F3. Next prod apply destroys the uploads bucket (High)	3
F4. Local state on one laptop (High)	3
F5. No managed encryption, no versioning in prod (High)	4
F6. Copy-pasted environments have drifted (Medium)	4
F7. Unpinned provider and Terraform version (Medium)	4
F8. Hardcoded values and no validation (Medium)	4
F9. Unmanaged access log bucket (Medium)	4
F10. No tags (Low)	5
Repair order	5
Out of scope	5
How to reproduce	5

This sample uses a fictional client, account and names throughout. The review covers **before/** in this repository. The evidence consists of actual output from the named tools, executed against that code; plans also use a local AWS emulator (moto). The account number 111122223333 is an AWS documentation example ID.

Summary

Harbor Goods manages product image storage through separate **dev** and **prod** Terraform folders, with both states stored on a single engineer's laptop. The review identified **10 findings: 2 critical, 3 high, 4 medium, 1 low**. Both critical exposures are active: anyone can read and list the prod uploads bucket, and the application role has unrestricted account permissions. F3 presents the most immediate operational risk. A resource rename without a **moved** block means the next **terraform apply** in **prod** would attempt to destroy the uploads bucket.

The repairs in **after/** address all ten findings with **0 resources destroyed**. The complete prod repair produces this plan: Plan: 1 to import, 12 to add, 7 to change, 0 to destroy.

Tool	before/	after/
Checkov 3.3.19 (Terraform)	25 passed, 47 failed	178 passed, 0 failed , 21 skipped with inline reasons
tfint 0.61.0, preset all + AWS ruleset 0.49.0	12 issues	0 issues
terraform validate	valid	valid
terraform test	none exist	9 runs, 9 passed

Scope and method

- **In scope:** review coverage includes the Terraform in `before/dev` and `before/prod`, prod's local state, and every resource referenced by that state.
- **Method:** diagnosis used read-only operations: static checks with Checkov, tfint and `terraform validate`; code inspection; `terraform plan` against existing state; and `terraform state list`. Diagnosis applied no changes. The planning CI role has `ReadOnlyAccess`, explicit denials for object, parameter and secret reads, and write permission limited to Terraform lock files (ADR 0001).
- **Ranking:** severity combines likelihood and impact. Critical covers current exploitability or data loss on the next apply; high covers an outage or data loss one mistake away; medium covers conditions that increase incident likelihood or recovery time; low covers hygiene.

Findings

ID	Risk	Finding	Where
F1	Critical	Uploads bucket is public: public-read ACL, public bucket policy, Block Public Access off	<code>before/*/main.tf</code>
F2	Critical	Application role policy allows * on *	<code>before/*/main.tf</code>
F3	High	A rename without <code>moved</code> makes the next prod apply destroy the uploads bucket	<code>before/prod/main.tf:17-20</code>
F4	High	State is a local file on one laptop: no locking, no history, one copy	both folders
F5	High	No KMS encryption, no versioning in prod, no lifecycle rules	<code>before/*/main.tf</code>
F6	Medium	Environments are copy-pasted and have drifted apart	<code>before/dev</code> vs <code>before/prod</code>
F7	Medium	Provider and Terraform versions unpinned, no lock file	<code>before/*/main.tf:5-11</code>
F8	Medium	Hardcoded names, region and ARNs; no input validation	both folders
F9	Medium	Access log bucket created by hand and unmanaged; no access logging	AWS console
F10	Low	No resource has an owner or cost center tag	both folders

F1. Uploads bucket is public (Critical)

- **Where:** ownership, public access block, ACL and policy settings in `before/prod/main.tf:22-66`; `before/dev` repeats these settings.

- **Evidence:**

```
Check: CKV_AWS_20: "S3 Bucket has an ACL defined which allows public READ access." FAILED /prod/main.tf:18-20
Check: CKV_AWS_53..56: block_public_acls / block_public_policy / ignore_public_acls / restrict_public_buckets FAILED /prod/main.tf:31-38
Check: CKV_AWS_70: "Ensure S3 bucket does not allow an action with any Principal" FAILED /prod/main.tf:50-66
Check: CKV2_AWS_65: "Ensure access control lists for S3 buckets are disabled" FAILED /prod/main.tf:22-28
```

- **Risk:** the `public-read` bucket ACL allows anyone to enumerate all keys as well as retrieve images through known URLs. Listing therefore exposes uploads intended to remain private, including invoices and returns photos.
- **Fix:** enable Block Public Access, disable ACLs through `BucketOwnerEnforced`, and limit the bucket policy to denying non-TLS requests. Immediately before applying, set the ACL to `private`: S3 will not disable ACLs while a public grant remains (runbook step 5). Serve public images through CloudFront with origin access control, which remains outside this scope as detailed below.

F2. Application role can do anything (Critical)

- **Where:** the role policy appears in `before/prod/main.tf:83-96` and `before/dev/main.tf:97-110`.
- **Evidence:**

```

Check: CKV_AWS_62: "Ensure IAM policies that allow full "*** administrative privileges are not created"  FAILED
Check: CKV_AWS_286: "Ensure IAM policies does not allow privilege escalation"                      FAILED
Check: CKV2_AWS_40: "Ensure AWS IAM policy does not allow full IAM privileges"                  FAILED
... 6 more IAM checks FAILED for aws_iam_role_policy.app

```
- **Risk:** code execution on an instance using this role gives an attacker control of the entire account, including permission to create admin users. A vulnerable dependency or SSRF targeting the metadata endpoint can provide that access.
- **Fix:** restrict the module policy to six actions: `s3:ListBucket` for the bucket; `s3:GetObject`, `s3:PutObject` and `s3>DeleteObject` for its objects; and `kms:Decrypt` and `kms:GenerateDataKey` for its key. Tests reject the reintroduction of `*`. Retaining the policy name allows an in-place update without replacement.

F3. Next prod apply destroys the uploads bucket (High)

- **Where:** code at `before/prod/main.tf:17-20` names the resource `uploads`, replacing `uploads_bucket`, while state still records `aws_s3_bucket.uploads_bucket`.
- **Evidence:** running `terraform plan` from `before/prod` against that folder's state produces:

```

# aws_s3_bucket.uploads will be created
# aws_s3_bucket.uploads_bucket will be destroyed
# (because aws_s3_bucket.uploads_bucket is not in configuration)
# aws_s3_bucket_acl.uploads must be replaced
# aws_s3_bucket_ownership_controls.uploads must be replaced
# aws_s3_bucket_policy.uploads must be replaced
# aws_s3_bucket_public_access_block.uploads must be replaced
Plan: 5 to add, 0 to change, 5 to destroy.

```
- **Risk:** the existing bucket occupies the name required by the new bucket, preventing creation; its objects also prevent deletion. The apply halts after completing some of the ten operations, leaving prod inconsistent with state. If `force_destroy` were enabled, as it is in dev, deletion would complete and remove every uploaded object.
- **Why High, not Critical:** the bucket's objects make the delete fail, so the apply halts before any data is lost.
- **Fix:** suspend applies in prod pending migration. The move chain in `after/envs/prod/moved.tf` is `uploads_bucket -> uploads -> module.storage.aws_s3_bucket.this`. The plan gate rejects PRs that would delete or replace a bucket (ADR 0005).

F4. Local state on one laptop (High)

- **Where:** both folders omit a `backend` block and consequently keep state in a working-directory `terraform.tfstate` file.
- **Evidence:** code inspection identifies this issue, which neither `tflint` nor Checkov reports. Only the engineer's laptop can run `terraform state list` successfully.
- **Risk:** simultaneous applies by two people corrupt state. Losing or wiping the laptop removes Terraform's record of managed resources, with no historical state available for rollback. Every resource attribute is also stored as plain text in the file.
- **Fix:** give each root an S3 backend using `use_lockfile = true`, without a DynamoDB table. Create the versioned, KMS-encrypted, TLS-only state bucket through `after/bootstrap` (ADR 0002), then follow the state migration runbook in `migration/`.

F5. No managed encryption, no versioning in prod (High)

- **Where:** `before/prod/main.tf:18-20` lacks versioning, although `before/dev` configures it.
- **Evidence:**

```
Check: CKV_AWS_145: "Ensure that S3 buckets are encrypted with KMS by default"          FAILED dev and prod
Check: CKV_AWS_21:  "Ensure all data stored in the S3 bucket have versioning enabled"    FAILED prod only
Check: CKV2_AWS_61: "Ensure that an S3 bucket has a lifecycle configuration"          FAILED dev and prod
```
- **Risk:** prod cannot recover an object overwritten or deleted by either a bug or a person. With default SSE-S3 encryption, no key policy is available to restrict or audit decryption access.
- **Fix:** configure SSE-KMS using a rotating customer managed key, with bucket keys to reduce KMS request costs. Enable versioning and a lifecycle policy that removes old versions after 90 days and aborts stale multipart uploads.

F6. Copy-pasted environments have drifted (Medium)

- **Where:** the environment definitions in `before/dev/main.tf` and `before/prod/main.tf` differ.
- **Evidence:** `diff before/dev/main.tf before/prod/main.tf` shows `force_destroy = true`, an `aws_s3_bucket_versioning` resource and a `var.env` variable in dev. Prod lacks all three and contains the resource rename described in F3.
- **Risk:** changes can reach only one environment, as already happened with versioning. Dev consequently ceases to test prod's configuration.
- **Fix:** consolidate the implementation in `after/modules/app-storage`, with two thin roots supplying only environment differences (ADR 0003). Expose `force_destroy` as an input whose validation prohibits enabling it for prod.

F7. Unpinned provider and Terraform version (Medium)

- **Where:** version declarations are in `before/prod/main.tf:5-11` and `before/dev/main.tf:5-11`; both directories lack `.terraform.lock.hcl`.
- **Evidence:**

```
before/prod/main.tf:7:11: Warning - Missing version constraint for provider "aws" in `required_providers`
before/prod/main.tf:5:1: Warning - terraform "required_version" attribute is required
```
- **Risk:** running `terraform init` on a fresh machine selects the latest available AWS provider. A major release may alter resource behavior, causing an unexpected plan during an otherwise unrelated change.
- **Fix:** set `required_version = ">= 1.11.0, < 2.0.0"` and `aws ~> 6.66` in each root. Commit lock files containing Linux and macOS hashes, and use Dependabot to submit provider upgrades through reviewed PRs.

F8. Hardcoded values and no validation (Medium)

- **Where:** both folders embed bucket names, role names, the region and the policy JSON's bucket ARN; variable `"env"` appears at `before/dev/main.tf:18`.
- **Evidence:**

```
before/dev/main.tf:18:1: Warning - `env` variable has no type (terraform_typed_variables)
before/dev/main.tf:18:1: Notice - `env` variable has no description (terraform_documented_variables)
```
- **Risk:** assigning `env = "production"` would produce a separate, unconfigured resource set. Manually changing an ARN can silently direct the policy to an incorrect bucket.
- **Fix:** derive names from validated `name_prefix` and `environment` inputs and obtain ARNs from resource attributes. Validate tags, retention periods and the trusted service as inputs as well.

F9. Unmanaged access log bucket (Medium)

- **Where:** the account contains `harbor-prod-access-logs`, but no Terraform configuration declares it.
- **Evidence:** `aws s3api list-buckets` includes the bucket, whereas `terraform state list` does not. Both uploads buckets fail CKV_AWS_18 "Ensure the S3 bucket has access logging enabled".
- **Risk:** the bucket's configuration and permitted writers are unknown. Without an access trail for F1's public bucket, the extent of that exposure cannot be established retrospectively.

- **Fix:** adopt the prod bucket through an `import` block in `after/envs/prod/imports.tf`. The module configures versioning, retention, a policy allowing only log delivery, and SSE-S3, the sole encryption supported by S3 log delivery. It also enables server access logging on the uploads bucket.

F10. No tags (Low)

- **Where:** resources throughout both folders lack `tags` arguments, and neither provider defines `default_tags`.
- **Evidence:** executing `grep -c tags before/*/main.tf` yields 0 for each file.
- **Risk:** storage costs cannot be allocated by environment or team, and incident responders have no named resource owner.
- **Fix:** require `Owner` and `CostCenter` as module inputs, with validation rejecting their absence. The module supplies `Environment`; each root supplies `ManagedBy`, `Repository` and `Stack` through `default_tags`.

Repair order

Use a separate pull request and plan for each step, completing review and apply before proceeding.

1. **Freeze prod applies** until step 5 to address F3. Communicate the freeze to the team; this step requires no code edit.
2. **Bootstrap the state backend** for F4 by applying `after/bootstrap` and migrating its own state into the bucket it creates.
3. **Migrate prod and dev state to S3** for F4 using runbook steps 2-4. Confirm that the migration leaves the plan unchanged.
4. **Agree how product images are served** after direct bucket access becomes private: place CloudFront in front or announce a URL change (F1). Prod cannot proceed to step 5 until this decision is made; dev can.
5. **Refactor into the module with moved, removed and import blocks** to resolve F3, F6, F7, F8 and F9. Require `0 to destroy` at the plan gate. Include F1, F2, F5 and F10 in this PR because they update these same resources in place. Separating those repairs would require initially reproducing the insecure settings in the new module. Apply dev first, then prod after dev applies cleanly.

Out of scope

- Delivering public product images through CloudFront with origin access control remains separate; F1 removes the bucket's direct public access.
- Upload replication across regions and a disaster-recovery target remain excluded; Checkov CKV_AWS_144 carries an inline skip explaining this exclusion.
- Bucket event notifications (CKV2_AWS_62) remain deferred until there is a consumer.
- Replacing the app's EC2 role with a workload identity, along with work beyond the two reviewed folders, falls outside this review.
- Rotating any credentials that may have been read through F1 or used through F2. Recommended as an immediate client action; it needs CloudTrail history that this review did not have.

How to reproduce

All procedures above work offline. Run `make findings` to verify that scanner results still match exactly the findings cited here and stored in `report/evidence/`. Run `make demo` to replay F3's plan, the migration and the `0 to destroy` plan using moto.

```
make findings    # before/: 47 failed Checkov checks and 12 tfLint issues, as recorded; after/: none
make demo       # F3 plan, state migration, refactor plan, plan gate, apply, clean re-plan
```

This is a fictional sample: Harbor Goods, its AWS account and all names used here are invented. Results were obtained by executing the named tools against this repository's `before/` code and do not describe an actual company.